CS 213，2002 年秋季实验作业 L5：编写自己的 Unix Shell 分配：10 月 24 日，截止：10 月 31 日 星期四，晚上 11:59

哈里·鲍维克（bovik@cs.cmu.edu）是这个作业的主要负责人。

引言

这个作业的目的是更熟悉进程控制和信号量的概念。你将通过编写一个支持作业控制的简单 Unix shell 程序来实现这一点。

物流

你可以与最多两人组成小组来完成本次作业的问题。唯一的“提交”方式将是电子提交。任何关于作业的说明和修订都会发布在课程网页上。

分发说明

> 特定站点说明：在此处插入一段解释教师将如何向学生分发 shlab-handout.tar 文件的段落。
> 以下是我们在 CMU 使用的说明。

首先将文件 shlab-handout.tar 复制到你计划进行工作的受保护目录（实验室目录）中。然后执行以下操作：

- 输入命令 tar xvf shlab-handout.tar 来解压 tar 文件。
- 输入命令 make 来编译和链接一些测试例程。
- 在 tsh.c 文件顶部的主注释中输入你的团队成员姓名和 Andrew ID。

查看 tsh.c（微型 shell）文件，你会看到它包含了一个简单 Unix shell 的功能框架。为了帮助你开始，我们已经实现了那些不太有趣的函数。你的任务

是要完成下面列出的剩余空函数。为了给你一个参考，我们在参考解决方案中列出了每个函数的大约代码行数（其中包含大量注释）。

- eval：主程序，用于解析和解释命令行。[70 行]
- builtin cmd：识别并解释内置命令：quit、fg、bg 和 jobs。[25 行]
- do bgfg: 实现 bg 和 fg 内置命令。[50 行]
- waitfg: 等待前台任务完成。[20 行]
- sigchld 处理器：捕获 SIGCHILD 信号。[80 行]
- sigint 处理器：捕获 SIGINT (ctrl-c) 信号。[15 行]
- sigtstp 处理器：捕获 SIGTSTP (ctrl-z) 信号。[15 行]

每次你修改 tsh.c 文件时，输入 make 来重新编译它。要运行你的 shell，在命令行中输入 tsh：

```
unix> ./tsh
tsh> [在这里输入命令到你的 shell]
```

Unix Shell 概述

一个 shell 是一个交互式命令行解释器，它代表用户运行程序。shell 会反复打印提示符，等待在标准输入(stdin)上输入命令行，然后根据命令行的内容执行某些操作。
命令行是一系列由空白字符分隔的 ASCII 文本单词。命令行的第一个单词要么是内置命令的名称，要么是可执行文件的路径名。其余的单词是命令行参数。如果第一个单词是内置命令，shell 会立即在当前进程中执行该命令。否则，该单词被认为是可执行程序的路径名。在这种情况下，shell 会创建一个子进程，然后在子进程的上下文中加载并运行程序。由解释单个命令行创建的子进程统称为一个作业。通常，一个作业可以由多个通过 Unix 管道连接的子进程组成。
如果命令行以一个与号“&”结束，那么该任务将在后台运行，这意味着外壳程序不会等待任务结束就打印提示符并等待下一个命令行。否则，该任务将在前台运行，这意味着外壳程序会等待任务结束才等待下一个命令行。因此，在任何时刻，最多只有一个任务可以在前台运行。然而，任意数量的任务都可以在后台运行。

例如，输入命令行

```
tsh> jobs
```

会使外壳程序执行内置的 jobs 命令。输入命令行

```
tsh> /bin/ls -l -d
```

在前景运行 ls 程序。按照惯例，当程序开始执行其主例程时

```
int main(int argc, char *argv[])
```

argc 和 argv 参数具有以下值：

- argc == 3,
- argv[0] == “/bin/ls”，
- argv[1]== “-l”，
- argv[2]== “-d”。

或者，在命令行中输入

```
tsh> /bin/ls -l -d &
```

可以在后台运行 ls 程序。

Unix shell 支持作业控制的概念，这允许用户在后台和前台之间切换作业，并改变作业中进程的状态（运行、停止或终止）。输入 ctrl-c 会导致向前台作业中的每个进程发送 SIGINT 信号。SIGINT 的默认动作是终止进程。类似地，输入 ctrl-z 会导致向前台作业中的每个进程发送 SIGTSTP 信号。SIGTSTP 的默认动作是将进程置于停止状态，直到收到 SIGCONT 信号时才会唤醒。Unix shell 还提供各种内置命令来支持作业控制。例如：

- 任务：列出正在运行和已停止的后台任务。
- bg <任务>：将已停止的后台任务改为正在运行的后台任务。
- fg <任务>：将已停止或正在运行的后台任务改为前台运行。
- kill : 终止一个作业。

tsh 规范

您的 tsh shell 应该具有以下功能：

- 提示符应该是字符串“tsh> ”。

· 用户输入的命令行应包含一个名称和零个或多个参数，所有内容通过一个或多个空格分隔。如果名称是内置命令，则 tsh 应立即处理它并等待下一个命令行。否则，tsh 应假定名称是可执行文件的路径，它加载并在此初始子进程的上下文中运行（在此上下文中，术语作业指代这个初始子进程）。

- tsh 无需支持管道（|）或 I/O 重定向（<和>）。
- 按下 ctrl-c (ctrl-z) 应该向当前前台作业以及该作业的任何子作业（例如，它所分叉的任何子进程）发送 SIGINT (SIGTSTP) 信号。如果没有前台作业，那么该信号不应产生任何效果。
- 如果命令行以 ampersand (&) 结尾，tsh 应在后台运行该作业。
  否则，应在前台运行该作业。
- 每个作业可以通过进程 ID (PID) 或作业 ID (JID) 来识别，JID 是 tsh 分配的正整数。在命令行中，JID 应使用前缀 '%' 表示。例如，“%5” 表示 JID 5，而 “5” 表示 PID 5。（我们已经为你提供了所有用于操作作业列表的例程。）
- tsh 应该支持以下内置命令：
  – 退出命令终止 shell。
  – 任务命令列出所有后台任务。
  – bg <任务> 命令通过发送 SIGCONT 信号重启 <任务>，然后将其在后台运行。<任务> 参数可以是 PID 或 JID。
  – fg 命令通过发送 SIGCONT 信号重启 ，然后在前景中运行它。 参数可以是 PID 或 JID。
- tsh 应该回收它所有的僵尸子进程。如果任何作业因为收到它没有捕获的信号而终止，tsh 应该识别这个事件，并打印包含作业的 PID 和违规信号描述的消息。

检查你的工作

我们提供了一些工具来帮助你检查工作。

参考解决方案。Linux 可执行文件 tshref 是 shell 的参考解决方案。运行此程序以解决你对 shell 应如何行为的任何疑问。你的 shell 应输出与参考解决方案完全相同的输出（当然，PID 除外，它会在每次运行时变化）。
Shell 驱动程序。sdriver.pl 程序作为子进程执行一个 shell，根据跟踪文件发送命令和信号，并捕获和显示 shell 的输出。使用-h 参数来了解 sdriver.pl 的用法：

```
unix> ./sdriver.pl -h 用法：sdriver.pl [-hv] -t <跟踪文件> -s -a <参数> 选项：
-h 打印此消息 -v 更详细 -t <跟踪文件> 跟踪文件 -s 要测试的 shell 程序 -a <参数>
shell 参数 -g 为自动评分器生成输出
```

我们还提供了 16 个跟踪文件（trace{01-16}.txt），你将使用它们与 shell 驱动程序一起测试你的 shell 的正确性。编号较小的跟踪文件执行非常简单的测试，而编号较大的测试则更复杂。

您可以在您的 shell 上运行 shell 驱动程序，使用跟踪文件 trace01.txt（例如），通过键入：

```
unix> ./sdriver.pl -t trace01.txt -s ./tsh -a "-p"
```

（-a "-p"参数告诉您的 shell 不要发出提示），或者

```
unix> make test01
```

同样地，要比较你的结果与参考 shell，你可以通过输入以下命令在参考 shell 上运行跟踪驱动程序：

```
unix> ./sdriver.pl -t trace01.txt -s ./tshref -a "-p"
```

或者

```
unix> make rtest01
```

供您参考，tshref.out 给出了参考解在所有比赛中的输出。这可能比您手动在所有跟踪文件上运行 shell 驱动器更方便。
跟踪文件的好处在于，它们生成的输出与您在交互模式下运行 shell 时得到的输出相同（除了一个标识跟踪的初始注释）。例如：

```
bass> make test15 ./sdriver.pl -t trace15.txt -s ./tsh -a
"-p" # # trace15.txt - 将所有内容整合在一起 # tsh> ./bogus
./bogus: 命令未找到。 tsh> ./myspin 10 任务（9721）被信号 2
终止 tsh> ./myspin 3 & [1] (9723) ./myspin 3 & tsh>
./myspin 4 &
```

```
/myspin 4 & [2] (9725) ./myspin 4 & tsh> jobs
[1] (9723) 运行中 ./myspin 3 & [2] (9725) 运行
中 ./myspin 4 & tsh> fg %1 任务[1] (9723) 被信
号 20 终止 tsh> jobs [1] (9723) 已停止 ./myspin
3 & [2] (9725) 运行中 ./myspin 4 & tsh> bg %3
%3: 无此任务 tsh> bg %1 [1] (9723) ./myspin 3 &
tsh> jobs [1] (9723) 运行中 ./myspin 3 & [2]
(9725) 运行中 ./myspin 4 & tsh> fg %1 tsh> quit
bass>
```

提示

- 仔细阅读教科书第 8 章（异常控制流）的每一个字。
- 使用跟踪文件来指导你的 shell 开发。从 trace01.txt 开始，确保你的 shell 产生与参考 shell 完全相同的输出。然后继续到 trace02.txt，依此类推。
- waitpid、kill、fork、execve、setpgid 和 sigprocmask 函数将非常有用。waitpid 的 WUNTRACED 和 WNOHANG 选项也会很有用。
- 当你实现你的信号处理器时，务必使用 kill 函数的参数中用“-pid”代替“pid”，将 SIGINT 和 SIGTSTP 信号发送给整个前台进程组。
  sdriver.pl 程序会测试这个错误。
- 作业中一个棘手的部分是决定在 `waitfg` 和 `sigchld` 处理器函数之间如何分配工作。我们推荐以下方法：
  - 在 waitfg 中，在 sleep 函数周围使用一个忙循环。
  - 在 sigchld 处理程序中，使用一次 waitpid 调用。

  虽然其他解决方案也是可能的，例如在 waitfg 和 sigchld 处理程序中都调用 waitpid，但这些可能会非常令人困惑。在处理程序中完成所有回收工作更简单。
- 在 eval 中，父进程必须在它 fork 子进程之前使用 sigprocmask 来阻塞 SIGCHLD 信号，然后再次使用 sigprocmask 来取消阻塞这些信号，在它通过调用 addjob 将子进程添加到作业列表后也是如此。由于子进程会继承父进程的阻塞向量，因此子进程必须确保在执行新程序之前取消阻塞 SIGCHLD 信号。

父进程需要以这种方式阻塞 SIGCHLD 信号，以避免竞争条件，即子进程在父进程调用 addjob 之前被 sigchld 处理程序回收（因此从作业列表中移除）。

- 像 more、less、vi 和 emacs 这样的程序会对终端设置做一些奇怪的事情。不要从你的 shell 中运行这些程序。坚持使用简单的基于文本的程序，比如 /bin/ls、/bin/ps 和 /bin/echo。

- 当你从标准 Unix shell 中运行 shell 时，你的 shell 正在前景进程组中运行。如果你的 shell 然后创建一个子进程，默认情况下那个子进程也会成为前景进程组的成员。由于按 ctrl-c 会向前景组中的每个进程发送 SIGINT，按 ctrl-c 会向你的 shell 以及你的 shell 创建的每个进程发送 SIGINT，这显然是不正确的。
  解决方法是：在 fork 之后但在 execve 之前，子进程应该调用 setpgid(0, 0)，这会将子进程放入一个新的进程组，其组 ID 与子进程的 PID 相同。这确保了前景进程组中只有一个进程，即你的 shell。当你按 ctrl-c 时，shell 应该捕获由此产生的 SIGINT，然后将其转发到适当的前景作业（或者更准确地说，包含前景作业的进程组）。

评估

您的分数将根据以下分布计算，最高分为 90 分：

80 正确性：16 个跟踪文件，每个 5 分。

10 风格分。我们希望您有良好的注释（5 分），并检查每个系统调用的返回值（5 分）。

您的解决方案 shell 将在 Linux 机器上测试其正确性，使用与您的实验目录中包含的相同 shell 驱动程序和跟踪文件。您的 shell 在这些跟踪文件上应产生与参考 shell 相同的输出，只有两个例外：

- PIDs 可以（并且会）不同。
- `trace11.txt`、`trace12.txt` 和 `trace13.txt` 中`/bin/ps` 命令的输出
  在每次运行时都会不同。然而，/bin/ps 命令输出中任何 mysplit 进程的运行状态应该是相同的。

提交说明

特定站点：在此处插入一段解释学生应如何提交他们的 tsh.c 文件的段落。以下是我们在 CMU 使用的说明。

· 确保你在 tsh.c 的头部注释中包含了你的姓名和 Andrew ID。

- 创建一个团队名称，形式如下：
  - “ID ”，其中 ID 是你的 Andrew ID，如果你是单独工作，或者
  - “ID+ID”其中 ID 是第一个团队成员的 Andrew ID，ID 是第二个团队成员的 Andrew ID。

  我们需要你按照这种方式创建团队名称，以便我们自动评分你的作业。
- 要提交你的 tsh.c 文件，请输入：

  ```
  make handin TEAM=teamname
  ```

  其中 teamname 是上面描述的团队名称。
- 提交后，如果你发现错误并想提交修订版，请输入

  ```
  make handin TEAM=teamname VERSION=2
  ```

  每次提交时，请继续递增版本号。
- 你应该通过查看

  ```
  /afs/cs.cmu.edu/academic/class/15213-f01/L5/handin
  ```

  来验证你的提交。在这个目录中，你有列表和插入权限，但没有读取或写入权限。

祝你好运！